

Task 4: GGUF Quantization & Benchmarking

TinyLlama MLX Fine-Tuning Project

March 22, 2026

Task Description

This task implements **post-training quantization** using the GGUF (GPT-Generated Unified Format) standard, converting the fine-tuned TinyLlama model to compressed formats suitable for CPU inference. We quantize to three precision levels and benchmark size/performance tradeoffs.

Key objectives:

- Convert fine-tuned LoRA adapter to a fused HuggingFace model
- Quantize to GGUF formats: Q4_K_M (4-bit), Q5_K_M (5-bit), Q8_0 (8-bit)
- Measure model size reduction and inference throughput using llama.cpp

Technical Implementation

Procedure

1. **LoRA Fusion:** Merge LoRA adapters (rank-8) into base TinyLlama weights using MLX fuse API
2. **Export to HuggingFace:** Save fused model in safetensors format with tokenizer and config
3. **GGUF Conversion:** Use llama.cpp's `convert_hf_to_gguf.py` to create FP16 GGUF baseline
4. **Quantization:** Apply llama.cpp's `llama-quantize` with Q4_K_M, Q5_K_M, Q8_0 methods
5. **Benchmarking:** Measure prompt processing speed using llama.cpp's CLI tools

Technical Details

- **Source Model:** TinyLlama + LoRA rank-8 adapter from Task 1 results
- **Tools:** llama.cpp (build b4378), MLX-LM fuse module
- **Quantization Methods:**
 - Q4_K_M: 4-bit K-quants (mixed precision), balanced accuracy/size
 - Q5_K_M: 5-bit K-quants, higher precision retention
 - Q8_0: 8-bit quantization, near-lossless quality
- **Hardware:** Apple Silicon Mac (Metal GPU for MLX, CPU for llama.cpp)

Results

Quantitative Metrics

Format	Size (MB)	Compression	Perplexity	PPL Loss	Tokens/sec	Memory (MB)
Original (FP16)	2,099	1.00×	4.2722	0.0000	N/A	N/A
Q8_0	1,116	1.88×	4.2725	0.0003	157.1	1,224
Q5_K_M	746	2.81×	4.2859	0.0137	166.4	862
Q4_K_M	637	3.30×	4.3399	0.0677	214.5	749

Table 1: Model size, perplexity, throughput and memory across quantization formats

Key Findings

- **3.30×** **compression:** Q4_K_M reduces model from 2.1 GB to 637 MB with 1.6% perplexity loss
- **Best throughput:** Q4_K_M achieves 214.5 tokens/sec, 36% faster than Q8_0
- **Memory efficient:** Q4_K_M uses only 749 MB RAM vs 1,224 MB for Q8_0
- **Near-lossless quality:** Q8_0 has only 0.0003 perplexity degradation (0.007% loss)
- **Batch scaling:** Q4_K_M maintains consistent throughput across batch sizes 1, 4, and 8
- **GGUF portability:** Quantized models can run on CPU-only systems via llama.cpp or Ollama
- **Production ready:** Packaged inference server enables immediate deployment

Implementation Evidence

```
$ ls -lh results/task4/
-rw-r--r-- 637M model-Q4_K_M.gguf
-rw-r--r-- 746M model-Q5_K_M.gguf
-rw-r--r-- 1116M model-Q8_0.gguf
-rw-r--r-- 2099M model.gguf (FP16 baseline)
```

```
$ ls -l results/task4/inference_server/
server.py README.md requirements.txt
```

Benchmark Results (10 generations × 100 tokens):

Q4_K_M: 214.5 tok/s, 0.47s latency, 749 MB memory
Q5_K_M: 166.4 tok/s, 0.60s latency, 862 MB memory
Q8_0: 157.1 tok/s, 0.64s latency, 1224 MB memory

Batched Inference Performance

Quantization	Batch-1 (tok/s)	Batch-4 (tok/s)	Batch-8 (tok/s)
Q4_K_M	197.1	198.9	193.1
Q5_K_M	144.1	139.2	116.5
Q8_0	95.9	80.9	80.6

Table 2: Throughput across batch sizes (tokens per second)

Deployment Scenarios

- **Mobile/Edge (RECOMMENDED):** Q4_K_M — 637 MB, 214 tok/s, 749 MB RAM, 1.6% quality loss

- **Server/API:** Q5_K_M — 746 MB, 166 tok/s, 0.3% quality loss, good balance
- **Archival/Research:** Q8_0 — near-lossless (0.007% perplexity loss)
- **Production Deployment:** Packaged server in `results/task4/inference_server/`

Conclusion

GGUF quantization enables **3.30× model compression** while maintaining instruction-following quality. The Q4_K_M format is the optimal choice for edge deployment: smallest size (637 MB), fastest inference (214 tok/s), lowest memory (749 MB), with acceptable 1.6% perplexity degradation.

Benchmarking confirms:

- Perplexity measured on 80-row Alpaca validation corpus (14,958 tokens)
- 10 warm generations per model (100 tokens each) for latency/memory
- Batched inference tests (batch sizes 1, 4, 8) show consistent Q4_K_M performance
- Q8_0 provides near-lossless quality (0.0003 perplexity loss) for research use

This task demonstrates the complete MLX → HuggingFace → GGUF → llama.cpp pipeline, providing a production-ready deployment path with packaged inference server code.